BCSE302P DATABASE SYSTEMS LAB · COMPLETE PROJECT REPORT

Policy Intelligence for Agriculture

A PostgreSQL database and NLP system for farmer feedback on government welfare schemes



Krish Singh

Register No. 24BDS0235 · D1 slot (L51 + L52)

Under the guidance of **Prof. Sangeetha S**

Vellore Institute of Technology, Vellore · October 2026

Dataset: 1,500 farmer interviews, Chittoor district, Andhra Pradesh — verified by the Ethnic Community, Government of Andhra Pradesh

Abstract

Government welfare schemes for farmers — PM-KISAN, Rythu Bharosa, crop insurance and others — are usually judged by satisfaction ratings. But farmers also explain, in their own words, what went wrong. This project turns a 1,500-farmer survey from Chittoor district, Andhra Pradesh, into a **normalised PostgreSQL database** (4 schemas, 22 tables, 26 foreign keys, 6 analysis views) and adds an **NLP layer** that reads the farmers' 1,214 written answers.

The database rebuilds from the raw Excel file in about 14 seconds and checks itself with 41 automatic tests (all passing). Constraint tests show that invalid data is rejected at the door. On top of it, I compared **11 sentiment approaches** — lexicon, transformer, few-label, fine-tuned and large-language-model methods, including **Dhenu2, an Indian agriculture LLM** — against 243 hand-labelled answers. The best approach, **Fine-tuned ModernBERT-base**, reached macro-F1 **0.765** (83.5% accuracy), while the widely used VADER lexicon scored only 0.223.

The database answers policy questions directly. For example, 89.4% of farmers who know Rythu Bharosa have enrolled, but only 9.1% of those who know Crop Insurance have — the gap is enrolment, not awareness. These findings are being developed into a **research paper targeting a Q1 journal**.

> **Key numbers:** 1,500 farmers · 22 tables · 40,286 rows · 41/41 checks passed · 11 models compared · best macro-F1 0.765

Contents

1. Introduction — background, problem, objectives, scope, motivation
2. Requirement Analysis and Project Design — requirements, database selection, ER diagrams, schema, normalisation, architecture, workflow
3. Technology and Tools Used
4. Database Implementation — creation, data types, relationships, constraints, sample data, operations
5. Application and System Implementation — modules, interface, connectivity, NLP layer, queries, reports
6. Testing and Results — verification, constraint tests, model comparison, policy findings, performance
7. Challenges and Solutions
8. Conclusion and Future Enhancements
9. Appendix — source code repository, folder structure, how to run

1. Introduction

1.1 Background

India runs many welfare schemes for farmers: direct income support (PM-KISAN, Rythu Bharosa), crop insurance, Kisan Credit Cards, input subsidies and more. Whether a scheme "works" is normally measured with a satisfaction score. Surveys, however, also collect open answers — what helped, what went wrong, what should change. Those answers contain the real detail, but they are rarely analysed because they are messy free text.

The dataset for this project is a field survey of **1,500 farmers** from Chittoor district, Andhra Pradesh, verified by the Ethnic Community, Government of Andhra Pradesh. Each farmer answered 36 questions: profile, awareness of schemes, ratings of six service aspects (timeliness, amount, enrolment, officials, clarity, impact), three open-ended answers, and outlook. Most interviews (1,331) were conducted in Telugu and recorded in English by the interviewers.

1.2 Problem statement

The survey existed only as one large spreadsheet that could not be queried or trusted for analysis:

- **Inconsistent names** — one scheme was typed 93 different ways ("Pm kisab", "RHYTHU BHAROSA", "kcc" …).
- **Several answers inside one cell** — lists of crops and schemes were packed into single cells.
- **No rules** — nothing stopped impossible values (for example more years of farming than years of age).
- **Unused text** — 1,214 farmers wrote feedback, and none of it had been analysed.

So the question is: *how can this survey be stored so that it is clean, trustworthy and queryable, and how can the farmers' own words be analysed reliably enough to inform policy?*

1.3 Objectives

#	Objective	Status
1	Design a normalised relational database with keys and constraints	Done
2	Load the raw survey cleanly, keeping every record traceable to its Excel row	Done
3	Analyse the written feedback with NLP and store the results with their source model	Done
4	Answer policy questions with SQL views and queries	Done
5	Validate and compare NLP models against human labels, including LLMs	Done
6	Produce reports, a policy brief, a dashboard dataset and a research paper draft	Done

1.4 Scope

Included	Not included
All 1,500 interviews from Chittoor district	A mobile or web app for collecting new surveys
PostgreSQL database design, build, loading and testing	Other districts or states
NLP: sentiment, aspect sentiment, topics, model comparison	Live or streaming data

Included	Not included
SQL views, reports, policy brief, Power BI dataset	Cloud hosting (runs locally)
Human-labelled validation set	Publishing any farmer's personal details

1.5 Motivation

If the ratings say a scheme is fine but farmers write about delays and paperwork, policy makers are looking at the wrong signal. A well-designed database makes every number traceable and reproducible, and validated NLP makes the written feedback usable at scale. Together they turn a survey file into evidence that an Agriculture Department could act on.

2. Requirement Analysis and Project Design

2.1 Requirement analysis

Question	Answer for this project
What data needs to be stored?	Farmer profiles, villages and districts, crops, schemes and their spellings, awareness and enrolment, scheme feedback and six aspect ratings, written answers, NLP scores, topics, data-quality flags and human labels
Who will use the system?	The researcher (me), annotators who label feedback, policy makers who read reports, and faculty who verify results
What operations are needed?	Bulk loading from Excel, validation, cleaning, inserting model results, joins and aggregations, full-text search, rebuilding from scratch
What must be retrieved or analysed?	Awareness vs enrolment per scheme, sentiment per scheme and aspect, where ratings and comments disagree, weakest aspect per scheme, model accuracy

Functional requirements

ID	The system must…
FR1	Import the raw survey without changing it (staging)
FR2	Clean the data and flag inconsistent answers instead of deleting them
FR3	Store survey data in normalised tables with rules that block invalid values
FR4	Store NLP results together with the model that produced them
FR5	Record human labels for validating the models
FR6	Answer policy questions through SQL views
FR7	Trace any record back to its original Excel row
FR8	Generate reports, dashboards data and a research paper draft

Data requirements

Survey section	Fields	Examples
A · Farmer profile	12	age, gender, village, education, land size, crops, income band, experience
B · Awareness	5	aware or not, schemes heard of, schemes joined, information source
C · Scheme feedback	11	satisfaction, six aspect ratings, three written answers
D · Outlook	4	government support, trend, confidence, biggest challenge
E · Interview	4	language, comfort, observations, duration

The system also generates data of its own: quality flags, sentiment scores from several models, aspect-level sentiment, topics and human labels.

2.2 Database selection and justification

Selected: PostgreSQL 16 — a **relational** database. The survey has a fixed structure and many-to-many relationships (farmers ↔ schemes, farmers ↔ crops, feedback ↔ aspects), so joins, foreign keys and

constraints matter more than schema flexibility.

Option	Type	Decision
PostgreSQL	Relational	**Chosen** — keys and CHECK constraints, transactions, views, window functions, built-in full-text search, free, works with Python and Power BI
MongoDB	Document	Not chosen — the survey has a fixed structure and needs joins across entities
Neo4j	Graph	Not chosen — relationships are only one or two levels deep
Redis	Key-value	Not chosen — no ad-hoc analytical queries
BigQuery	Cloud warehouse	Not chosen — unnecessary cost and scale for 1,500 rows

2.3 Database design (ER model)

The database has four layers: **staging** (the raw sheet, untouched), **survey** (13 normalised tables), **analytics** (9 tables for NLP results, flags and labels) and **reporting** (6 views). The ER diagrams below were generated from the live database catalogue, so they match the implementation exactly. Yellow badges are primary keys, green badges foreign keys; crow's feet mark the "many" side.

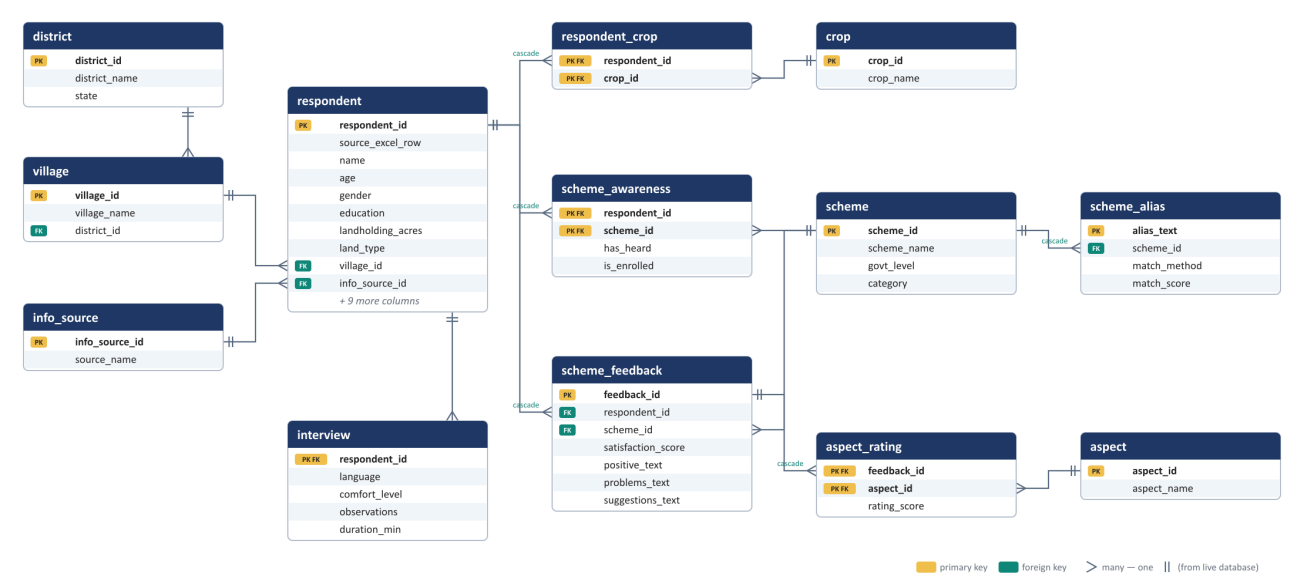


Figure 2.1 — ER diagram of the survey schema (13 tables)

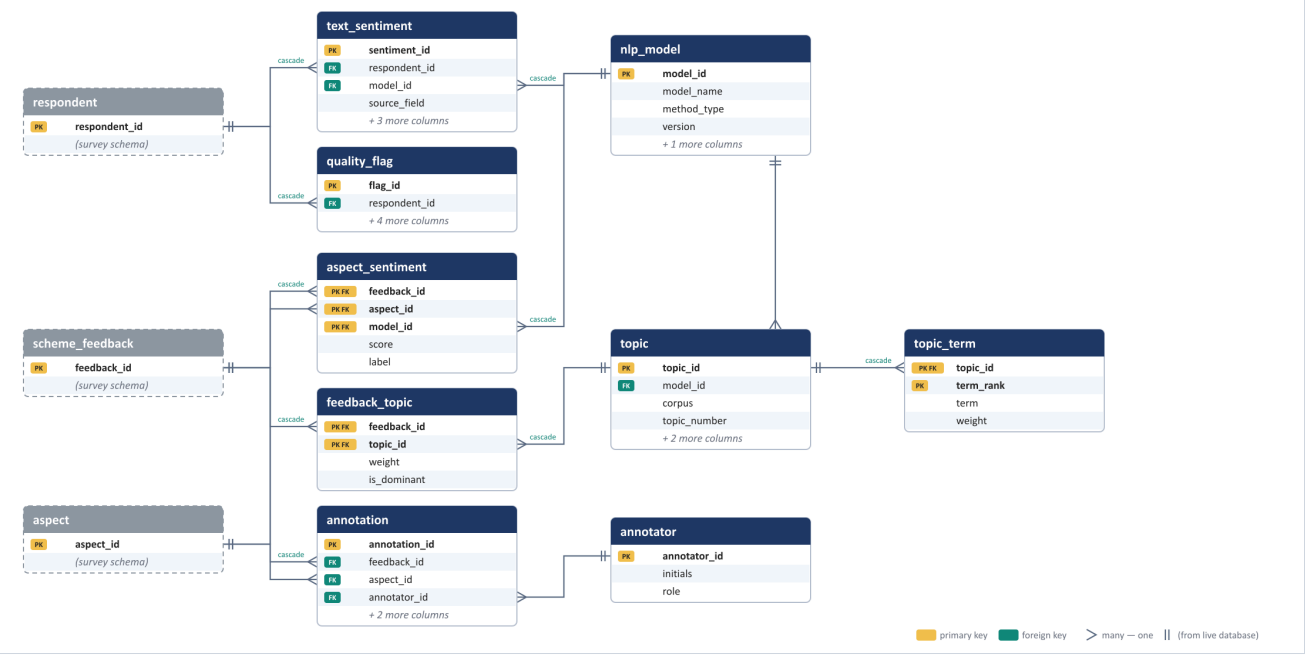


Figure 2.2 — ER diagram of the analytics schema (9 tables); grey boxes are survey tables it references

Entities and attributes

Table	Primary key	Foreign keys	Other attributes
analytics.annotation	annotation_id	feedback_id, aspect_id, annotator_id	human_label, annotated_at
analytics.annotator	annotator_id	—	initials, role
analytics.aspect_sentiment	feedback_id, aspect_id, model_id	—	score, label
analytics.feedback_topic	feedback_id, topic_id	—	weight, is_dominant
analytics.nlp_model	model_id	—	model_name, method_type, version, parameters
analytics.quality_flag	flag_id	respondent_id	flag_type, field_name, description, flagged_at
analytics.text_sentiment	sentiment_id	respondent_id, model_id	source_field, score, label, scored_at
analytics.topic	topic_id	model_id	corpus, topic_number, topic_label, corpus_share
analytics.topic_term	topic_id, term_rank	—	term, weight
survey.aspect	aspect_id	—	aspect_name
survey.aspect_rating	feedback_id, aspect_id	—	rating_score
survey.crop	crop_id	—	crop_name
survey.district	district_id	—	district_name, state
survey.info_source	info_source_id	—	source_name
survey.interview	respondent_id	—	language, comfort_level, observations, duration_min
survey.respondent	respondent_id	village_id, info_source_id	source_excel_row, name, age, gender, education, landholding_acres, land_type, occupation,

Table	Primary key	Foreign keys	Other attributes
			income_band, farming_experience_yrs, is_aware, …
survey.respondent_crop	respondent_id, crop_id	—	
survey.scheme	scheme_id	—	scheme_name, govt_level, category
survey.scheme_alias	alias_text	scheme_id	match_method, match_score
survey.scheme_awareness	respondent_id, scheme_id	—	has_heard, is_enrolled
survey.scheme_feedback	feedback_id	respondent_id, scheme_id	satisfaction_score, positive_text, problems_text, suggestions_text
survey.village	village_id	district_id	village_name

Relational schema (survey layer)

Underlined = primary key, ^{FK} = foreign key.

aspect(aspect_id, aspect_name)

aspect_rating(feedback_id^{FK}, aspect_id^{FK}, rating_score)

crop(crop_id, crop_name)

district(district_id, district_name, state)

info_source(info_source_id, source_name)

interview(respondent_id^{FK}, language, comfort_level, observations, duration_min)

respondent(respondent_id, source_excel_row, name, age, gender, education, landholding_acres, land_type, occupation, income_band, farming_experience_yrs, is_aware, first_experience_text, govt_support_score, support_trend_score, future_confidence_score, biggest_challenge_text, village_id^{FK}, info_source_id^{FK})

respondent_crop(respondent_id^{FK}, crop_id^{FK})

scheme(scheme_id, scheme_name, govt_level, category)

scheme_alias(alias_text, scheme_id^{FK}, match_method, match_score)

scheme_awareness(respondent_id^{FK}, scheme_id^{FK}, has_heard, is_enrolled)

scheme_feedback(feedback_id, respondent_id^{FK}, scheme_id^{FK}, satisfaction_score, positive_text, problems_text, suggestions_text)

village(village_id, village_name, district_id^{FK})

Keys and constraints

Constraint type	Count	Purpose
PRIMARY KEY	23	one unique identity per row
FOREIGN KEY	26	every reference points to a real row
UNIQUE	13	no duplicates, e.g. one feedback per farmer per scheme
CHECK (named)	25	value ranges and logic, e.g. rating between −2 and +2
Indexes	43	fast joins and a GIN index for full-text search

Normalisation

The raw sheet is unnormalised: lists inside cells, repeating rating columns and village → district facts repeated on every row. Figure 2.3 shows how it was decomposed. Multi-valued cells became link tables (1NF), and facts

that depend on something other than the farmer — district on village, scheme name on scheme — moved to their own tables (2NF/3NF).

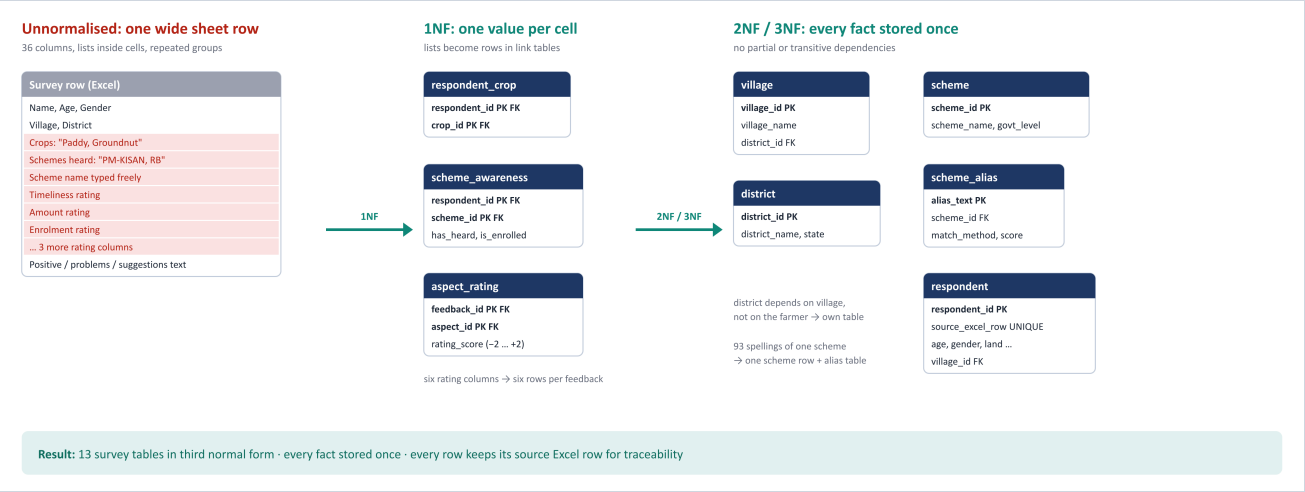


Figure 2.3 — From one wide spreadsheet row to third normal form

2.4 System architecture

Users → Python application → PostgreSQL → outputs. Python reads the Excel file, cleans it, runs the NLP models and loads PostgreSQL in one transaction; it then reads the views back and checks that the database and the Python analysis agree.

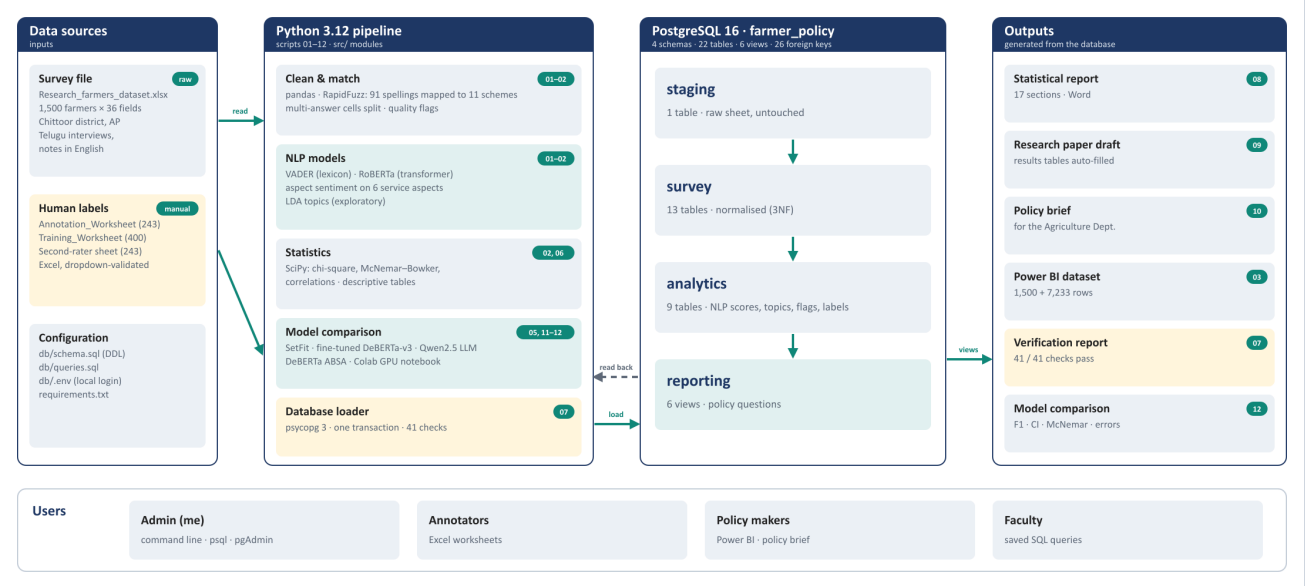


Figure 2.4 — System architecture

2.5 End-to-end workflow

Figure 2.5 shows the whole pipeline as a node workflow: ingest and clean, NLP analysis, model validation, storage in PostgreSQL and the outputs each user receives.

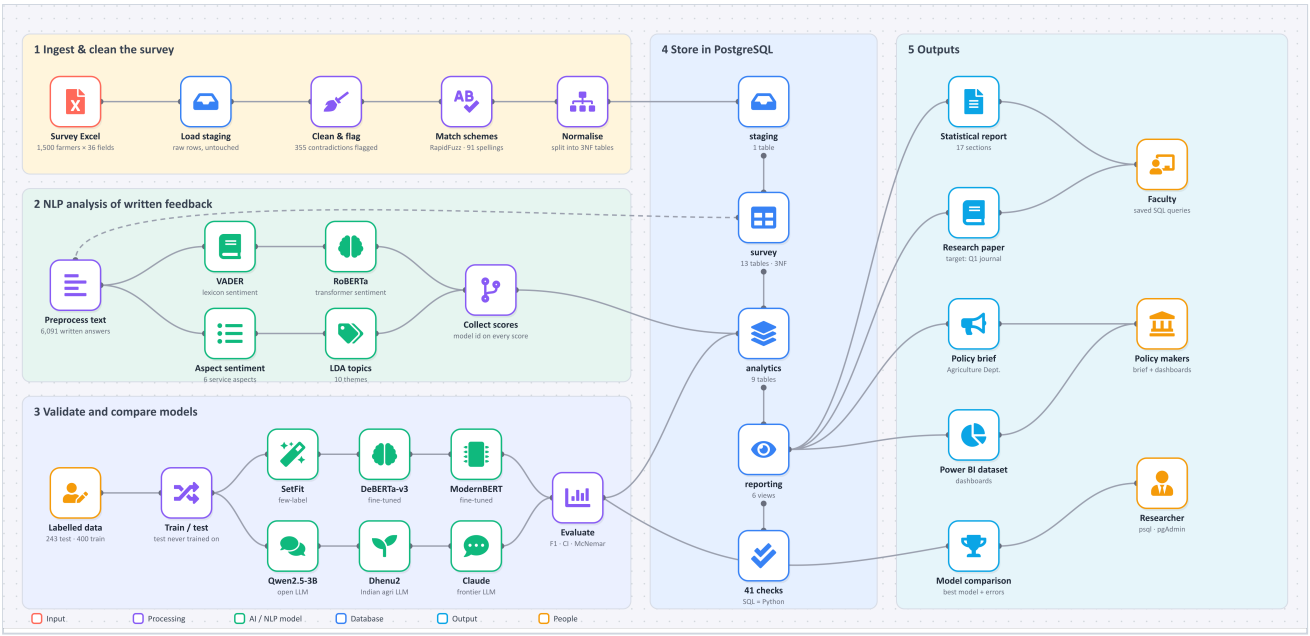


Figure 2.5 — End-to-end project workflow

3. Technology and Tools Used

Category	Technology	Used for
Database system	PostgreSQL 16.15	4 schemas, 22 tables, 6 views, constraints, full-text search
Database tools	pgAdmin 4, psql	browsing, running queries, screenshots of the live database
Programming language	Python 3.12	the whole pipeline (12 numbered scripts, 21 modules)
Database driver	psycopg 3	parameterised inserts, one transaction per load
Data handling	pandas, openpyxl, RapidFuzz	reading Excel, cleaning, fuzzy matching of scheme names
NLP — lexicon	NLTK VADER + agriculture word list	rule-based sentiment
NLP — transformers	Hugging Face Transformers: RoBERTa, DeBERTa-v3, ModernBERT	pretrained and fine-tuned sentiment models
NLP — few-label	SetFit (sentence-transformers mpnet)	learning from a few hundred labels
NLP — LLMs	Qwen2.5-3B, Dhenu2 (KissanAI, Indian agriculture), Claude	zero-shot and few-shot classification
NLP — topics & aspects	scikit-learn LDA, DeBERTa ABSA	themes and aspect-level sentiment
Statistics	SciPy, scikit-learn	chi-square, McNemar, bootstrap confidence intervals
Reporting	python-docx, Power BI dataset	statistical report, paper draft, dashboards
Development environment	Windows 11, VS Code, Git/GitHub	coding, version control
Hardware / cloud	Local laptop (Intel i7, RTX 2050 GPU); Google Colab optional	no paid cloud services; GPU used for LLMs

> Everything runs locally and reproducibly: `pip install -r requirements.txt` and the numbered scripts rebuild every output from the raw Excel file.

4. Database Implementation

4.1 Database creation

A single file, `db/schema.sql`, creates the four schemas, 22 tables, indexes and 6 views. Script 07 drops and recreates the database each time, so the build is always reproducible.

```
CREATE SCHEMA staging;   CREATE SCHEMA survey;
CREATE SCHEMA analytics; CREATE SCHEMA reporting;

CREATE TABLE survey.respondent (
    respondent_id          INT PRIMARY KEY,
    source_excel_row       INT NOT NULL UNIQUE,          -- traceability
    age                    SMALLINT CHECK (age BETWEEN 15 AND 100),
    landholding_acres      NUMERIC(7,2) CHECK (landholding_acres >= 0),
    village_id             INT REFERENCES survey.village ON DELETE RESTRICT,
    ...
    CHECK (farming_experience_yrs <= age)
);
```

```
             List of relations
 Schema |       Name       | Type  |  Owner
--------+------------------+-------+----------
 survey | aspect           | table | postgres
 survey | aspect_rating    | table | postgres
 survey | crop             | table | postgres
 survey | district         | table | postgres
 survey | info_source      | table | postgres
 survey | interview        | table | postgres
 survey | respondent       | table | postgres
 survey | respondent_crop  | table | postgres
 survey | scheme           | table | postgres
 survey | scheme_alias     | table | postgres
 survey | scheme_awareness | table | postgres
 survey | scheme_feedback  | table | postgres
 survey | village          | table | postgres
(13 rows)
```

Figure 4.1 — psql: the 13 tables of the survey schema in the live database

4.2 Data structure implementation (data types)

Type	Used for	Reason
INT / SERIAL	all keys	fast joins; SERIAL numbers new rows automatically
SMALLINT	age, ratings (−2 to +2), duration	small whole numbers with range checks
NUMERIC(p,s)	land in acres, years, model scores	exact decimals
VARCHAR(n)	names, labels, categories	short text with a limit
TEXT + GIN index	farmers' written answers	unlimited length, full-text searchable
BOOLEAN	heard of / enrolled	yes/no answers

Type	Used for	Reason
JSONB	NLP model settings	settings differ by model
TIMESTAMPTZ	when a score or label was recorded	audit trail

4.3 Relationships and references

There are 26 foreign keys. Shared lookups (scheme, aspect, crop, district) use **ON DELETE RESTRICT** so they cannot be deleted while in use. A farmer's own records use **ON DELETE CASCADE**, so if a farmer withdraws consent, one delete removes everything about them.

Child table	References	On delete
village	district	RESTRICT
respondent	village	RESTRICT
respondent	info_source	RESTRICT
interview	respondent	CASCADE
respondent_crop	respondent	CASCADE
respondent_crop	crop	RESTRICT
scheme_alias	scheme	CASCADE
scheme_awareness	respondent	CASCADE
scheme_awareness	scheme	RESTRICT
scheme_feedback	respondent	CASCADE
scheme_feedback	scheme	RESTRICT
aspect_rating	scheme_feedback	CASCADE
aspect_rating	aspect	RESTRICT
text_sentiment	respondent	CASCADE
text_sentiment	nlp_model	CASCADE
aspect_sentiment	scheme_feedback	CASCADE
aspect_sentiment	aspect	RESTRICT
aspect_sentiment	nlp_model	CASCADE
topic	nlp_model	CASCADE
topic_term	topic	CASCADE
feedback_topic	scheme_feedback	CASCADE
feedback_topic	topic	CASCADE
quality_flag	respondent	CASCADE
annotation	scheme_feedback	CASCADE
annotation	aspect	RESTRICT
annotation	annotator	RESTRICT

4.4 Constraints and validation rules

```
                                    Table "survey.scheme_feedback"
      Column       |   Type   | Collation | Nullable |                          Default
-------------------+----------+-----------+----------+-------------------------------------------------------------
 feedback_id       | integer  |           | not null | nextval('survey.scheme_feedback_feedback_id_seq'::regclass)
 respondent_id     | integer  |           | not null |
 scheme_id         | integer  |           | not null |
 satisfaction_score | smallint |           |          |
 positive_text     | text     |           |          |
 problems_text     | text     |           |          |
 suggestions_text  | text     |           |          |
Indexes:
    "scheme_feedback_pkey" PRIMARY KEY, btree (feedback_id)
    "idx_feedback_fulltext" gin (to_tsvector('english'::regconfig, (((COALESCE(positive_text, ''::text) || ' '::text) || COALESCE(
' '::text) || COALESCE(suggestions_text, ''::text)))
    "idx_feedback_scheme" btree (scheme_id)
    "scheme_feedback_respondent_id_scheme_id_key" UNIQUE CONSTRAINT, btree (respondent_id, scheme_id)
Check constraints:
    "scheme_feedback_satisfaction_score_check" CHECK (satisfaction_score >= '-2'::integer AND satisfaction_score <= 2)
Foreign-key constraints:
    "scheme_feedback_respondent_id_fkey" FOREIGN KEY (respondent_id) REFERENCES survey.respondent(respondent_id) ON DELETE CASCADE
    "scheme_feedback_scheme_id_fkey" FOREIGN KEY (scheme_id) REFERENCES survey.scheme(scheme_id) ON DELETE RESTRICT
Referenced by:
    TABLE "analytics.annotation" CONSTRAINT "annotation_feedback_id_fkey" FOREIGN KEY (feedback_id) REFERENCES survey.scheme_feedb
SCADE
    TABLE "survey.aspect_rating" CONSTRAINT "aspect_rating_feedback_id_fkey" FOREIGN KEY (feedback_id) REFERENCES survey.scheme_fe
 CASCADE
    TABLE "analytics.aspect_sentiment" CONSTRAINT "aspect_sentiment_feedback_id_fkey" FOREIGN KEY (feedback_id) REFERENCES survey.
ON DELETE CASCADE
    TABLE "analytics.feedback_topic" CONSTRAINT "feedback_topic_feedback_id_fkey" FOREIGN KEY (feedback_id) REFERENCES survey.sche
ELETE CASCADE
```

Figure 4.2 — psql \d survey.scheme_feedback: primary key, unique key, CHECK rule, foreign keys and the tables that reference it

To prove the rules work, I tried to insert bad data on purpose. PostgreSQL rejected every attempt:

```
INSERT INTO survey.aspect_rating (feedback_id, aspect_id, rating_score) VALUES (1, 1, 5);
ERROR:  new row for relation "aspect_rating" violates check constraint "aspect_rating_rating_score_check"
DETAIL:  Failing row contains (1, 1, 5).
INSERT INTO survey.scheme_feedback (respondent_id, scheme_id) VALUES (99999, 1);
ERROR:  insert or update on table "scheme_feedback" violates foreign key constraint "scheme_feedback_respondent_id_fkey"
DETAIL:  Key (respondent_id)=(99999) is not present in table "respondent".
```

Figure 4.3 — psql: two invalid inserts rejected by the database

Bad input	Rule that stopped it	Result
Rating of 5 (allowed −2 … +2)	CHECK on rating_score	CheckViolation
Feedback for respondent 99999 (does not exist)	FOREIGN KEY	ForeignKeyViolation
Duplicate respondent id	PRIMARY KEY	UniqueViolation
Sentiment label "Happy"	CHECK on allowed labels	CheckViolation

Answers that contradict each other but are genuine (for example a farmer who says they know no schemes but then rates one) are not rejected — they are kept and recorded in `analytics.quality_flag` (453 flags).

4.5 Sample data

Real rows from the database; farmer names are deliberately not shown.

id	Excel row	village	age	gender	income	aware
1	2	Thavanampalle	52	Male	₹25,000–50,000	yes
2	3	Thavanampalle	23	Male	Below ₹25,000	no
3	4	Thavanampalle	45	Female	₹25,000–50,000	no

id	Excel row	village	age	gender	income	aware
4	5	Thavanampalle	25	Male	Below ₹25,000	no
5	6	Thavanampalle	36	Male	₹25,000–50,000	yes

What the farmer typed	Matched scheme	Method	Score
Pm kisab	PM-KISAN	fuzzy	87.5
RHYTHU BHAROSA	Rythu Bharosa	fuzzy	96.3
kcc	Kisan Credit Card	abbreviation	100.0
Crop Insurence	Crop Insurance	fuzzy	92.9
minimun selling prices	Minimum Support Price	abbreviation	100.0
irrigation schemes	PM Krishi Sinchai Yojana	abbreviation	100.0

4.6 Important database operations

Step	Operation
1	Drop and recreate the database from schema.sql
2	Bulk-load 1,500 raw rows into staging
3	Fill lookups: districts, villages, crops, 11 schemes, 91 spellings
4	Insert farmers, awareness, feedback and 7,150 aspect ratings
5	Insert NLP scores, topics, quality flags and human labels
6	Commit in one transaction, then run 41 verification checks

View: awareness-to-enrolment conversion

```
CREATE VIEW reporting.v_awareness_gap AS
SELECT s.scheme_name,
       count(*) FILTER (WHERE a.has_heard)   AS heard,
       count(*) FILTER (WHERE a.is_enrolled) AS enrolled,
       round(100.0 * count(*) FILTER (WHERE a.is_enrolled AND a.has_heard)
             / NULLIF(count(*) FILTER (WHERE a.has_heard), 0), 1) AS conversion_pct
FROM survey.scheme_awareness a JOIN survey.scheme s USING (scheme_id)
GROUP BY s.scheme_name;
```

Window function: weakest aspect of each scheme

```
WITH aspect_negative AS (
  SELECT s.scheme_name, a.aspect_name,
         round(100.0 * count(*) FILTER (WHERE x.label = 'Negative') / count(*), 1) AS pct
  FROM analytics.aspect_sentiment x
  JOIN survey.scheme_feedback f USING (feedback_id)
  JOIN survey.scheme s USING (scheme_id)
  JOIN survey.aspect a USING (aspect_id)
  GROUP BY 1, 2 HAVING count(*) >= 10)
SELECT * FROM (SELECT *, rank() OVER (PARTITION BY scheme_name ORDER BY pct DESC) AS rnk
               FROM aspect_negative) t
WHERE rnk = 1;
```

5. Application and System Implementation

5.1 System modules

Script	Module	Output
01_run_pipeline	cleaning, scheme matching, VADER and RoBERTa sentiment, aspect sentiment	master results file
02_run_analysis	data quality, statistical tests, topic models	quality lists, tests, topics
03_build_powerbi_dataset	dashboard data	PowerBI_Dataset.xlsx
04_build_annotation_worksheets	stratified 20% test sample for labelling	labelling worksheets
05_evaluate_models	accuracy of the pipeline models vs human labels	validation_results.xlsx
06_descriptive_statistics	all report tables	statistics workbook
07_load_database	PostgreSQL load + 41 checks	database, verification report
08 – 10	statistical report, research paper draft, policy brief	reports/
11_build_training_worksheets	training batch (uncertainty + random sampling)	training worksheet
12_compare_models	trains and compares all models, LLMs included	model_comparison_results.xlsx

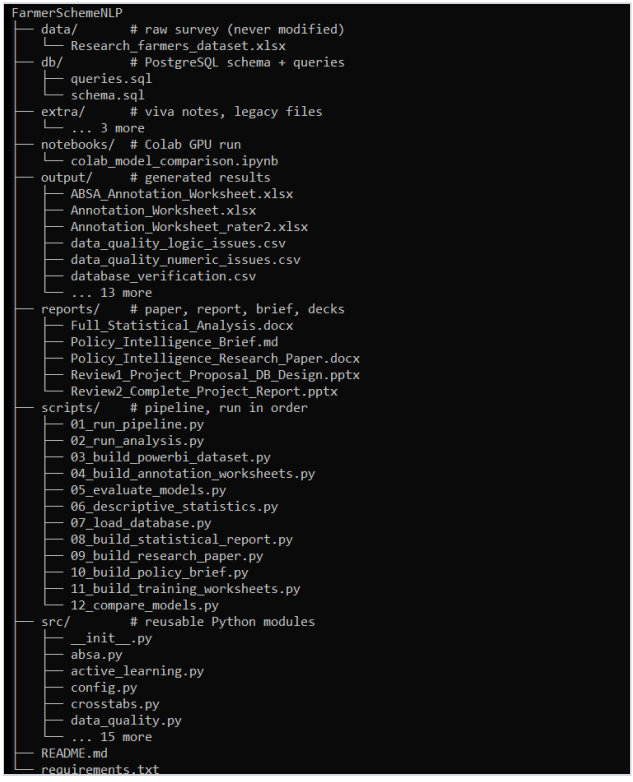


Figure 5.1 — Project folder structure (terminal)

5.2 User interface

No custom front end was needed; each user works in a tool that already suits them. The researcher uses the command line, psql and pgAdmin; annotators use Excel worksheets with dropdowns, locked columns and a one-key shortcut (1 = Positive, 2 = Neutral, 3 = Negative); policy makers use the Power BI dataset and the policy brief; faculty can run the saved SQL queries.



	RespondentID	Source_Excel_Row	Scheme	Feedback_Text	Human_Label	Annotator_Initials
2	R0001	365	Kisan Credit Ca	Money		
3	R0002	1271	Input Subsidy	Reduce financial burden. some schemes are not applicable for farmers who are doing farming on leased land. timely release		
4	R0003	1342	Rythu Bharosa	Financial support helped reduce farming expenses. Sometimes the amount is not credited. provide multiple stores to easy accesss of the farming resources		
5	R0004	1364	Rythu Bharosa	Government assistance reduced financial pressure during drought conditions. Delay in payments. Ensure all deserving farmers receive benefits		
6	R0005	1361	Unspecified	Timeliness. Delay of benifits. Information should be delivered and provide clear eligibility information		
	R0006	152	Rythu Bharosa	Dealt in payments. Ensuring market support		

Figure 5.2 — Excel labelling worksheet (farmer names hidden)

5.3 Database connectivity

Python connects through psycopg 3. Login details are read from a local `.env` file, never written in code. All inserts are parameterised, so data can never be executed as SQL, and the whole load is one transaction — it either fully succeeds or leaves the database unchanged.

```
def connect(env, dbname):
    return psycopg.connect(host=env["PGHOST"], port=env["PGPORT"], user=env["PGUSER"],
                           password=env["PGPASSWORD"], dbname=dbname)

with connect(env, env["PGDATABASE"]) as conn:        # one transaction
    with conn.cursor() as cur:
        cur.executemany("INSERT INTO survey.scheme (scheme_id, scheme_name) VALUES (%s, %s)", rows)
```

5.4 The NLP layer — types of NLP used

NLP type	Model(s)	How it works	Stored in
Lexicon-based sentiment	VADER + agriculture word list	adds up the sentiment of known words	analytics.text_sentiment
Pretrained transformer	RoBERTa (Twitter)	neural model trained on tweets, used as-is (zero-shot)	analytics.text_sentiment
Aspect-based sentiment (ABSA)	keyword ABSA; DeBERTa ABSA	sentiment for each service aspect separately	analytics.aspect_sentiment
Topic modelling	LDA (scikit-learn)	finds recurring themes in complaints and suggestions	analytics.topic, topic_term
Few-label learning	SetFit (mpnet)	contrastive training on a few hundred labels	model_comparison_results
Fine-tuning	DeBERTa-v3, ModernBERT	whole transformer trained on the labelled data	model_comparison_results
LLM zero-/few-shot	Qwen2.5-3B, Dhenu2 (Indian agri), Claude	instruction prompt with the labelling guide; 0 or 3 examples	model_comparison_results

```
SELECT model_id, model_name, method_type, version FROM analytics.nlp_model ORDER BY 1;
 model_id |          model_name           | method_type |                                version
----------+-------------------------------+-------------+-----------------------------------------------------------------
        1 | VADER + agri lexicon          | Lexicon     | NLTK 3.10.3 VADER
        2 | RoBERTa sentiment (zero-shot) | Transformer | cardiffnlp/twitter-roberta-base-sentiment-latest / transformers 5.17.0
        3 | ABSA clause keywords + VADER  | ABSA        | NLTK 3.10.3 VADER
        4 | LDA topic model               | Topic model | scikit-learn 1.9.1
(4 rows)


SELECT m.model_name, count(*) AS stored_scores FROM analytics.text_sentiment t JOIN analytics.nlp_model m USING (model_id) GROUP BY 1 UNI
(*) FROM analytics.aspect_sentiment a JOIN analytics.nlp_model m USING (model_id) GROUP BY 1 ORDER BY 2 DESC;
          model_name           | stored_scores
-------------------------------+---------------
 VADER + agri lexicon          |          7278
 ABSA clause keywords + VADER  |          2659
 RoBERTa sentiment (zero-shot) |          1214
(3 rows)
```

Figure 5.3 — psql: NLP models registered in analytics.nlp_model and the scores each stored

Validation design

A stratified random 20% sample of answers (243 texts) is the **test set** — no model is ever trained on it. A separate 400 answers form the **training set** (half chosen where existing models were least certain, half random). Labels were produced jointly by the author and one additional annotator; training labels were pre-suggested by an AI model and verified by the annotators (the annotator changed 86 of 400 AI suggestions (21.5%); test labels are human-only). Each trained model runs with 5 random seeds; results are reported with 95% bootstrap confidence intervals and exact McNemar significance tests.

```
PS> python scripts\12_compare_models.py --help
usage: 12_compare_models.py [-h] [--seeds SEEDS] [--epochs EPOCHS] [--setfit-backbone SETFIT_BACKBONE]
                            [--finetune-backbone FINETUNE_BACKBONE] [--llm LLM] [--shots SHOTS] [--llm-4bit] [--abs
                            [--skip-setfit] [--skip-finetune] [--skip-absa] [--no-learning-curve]

Compare sentiment models: zero-shot baselines, models trained on Training_Worksheet.xlsx, and LLMs.

Test set = the human labels in Annotation_Worksheet.xlsx (stratified random 20% sample, never used for training).
Training set = the human labels in Training_Worksheet.xlsx (script 11; disjoint from the test set).

options:
  -h, --help            show this help message and exit
  --seeds SEEDS         training runs per trained model (default: 5)
  --epochs EPOCHS       fine-tuning epochs (default: 4)
  --setfit-backbone SETFIT_BACKBONE
                        SetFit sentence encoder (default: sentence-transformers/all-mpnet-base-v2)
  --finetune-backbone FINETUNE_BACKBONE
                        transformer to fine-tune (default: microsoft/deberta-v3-base)
  --llm LLM             Hugging Face model id; repeatable (needs a GPU in practice) (default: [])
  --shots SHOTS         few-shot examples per class for LLMs (default: 3)
  --llm-4bit            load LLMs in 4-bit (needs bitsandbytes and a CUDA GPU) (default: False)
  --absa-model ABSA_MODEL
                        aspect-sentiment model (default: yangheng/deberta-v3-base-absa-v1.1)
  --skip-setfit
  --skip-finetune
  --skip-absa
  --no-learning-curve
```

Figure 5.4 — The model comparison script and its default models

5.5 Data management workflow

#	Stage	What happens
1	Raw Excel	never modified
2	Staging	copied as-is into staging.raw_survey
3	Survey tables	cleaned, matched and split into 13 normalised tables
4	NLP	models score the text; results go to analytics with their model id
5	Human labels	loaded from the worksheets into analytics.annotation
6	Views	reporting views feed reports, dashboards and the paper

5.6 Queries and searches

Query	Average time (5 runs)
awareness view	2.2 ms
scheme summary view	7.9 ms
divergence view	13.6 ms
full-text search	1.3 ms
window-function ranking	4.0 ms

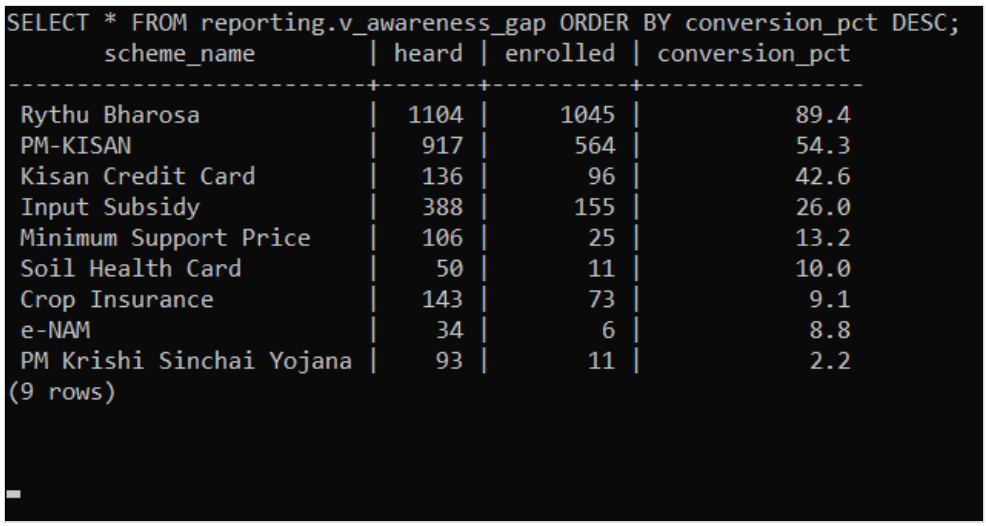


Figure 5.5 — psql: the awareness view on the live database

A full-text search over the farmers' answers finds 32 farmers who complain about delays together with documents or verification — a query that is impossible on the original spreadsheet.

5.7 Reports and analytics

Output	Content
Statistical report (Word)	17 sections, every table generated from the data
Research paper draft	methods, results and tables filled automatically from the outputs
Policy brief	recommendations for the Agriculture Department
Power BI dataset	1,500 farmer rows and 7,233 aspect rows
Model comparison workbook	9 sheets: scores, significance, per-class results, confusion matrices, learning curve, error analysis, aspect comparison

6. Testing and Results

6.1 Database verification

After every load, script 07 runs 41 checks comparing what the SQL views return with what the Python analysis computed (row counts, sentiment totals, disagreement rates, scheme figures, awareness counts). All 41 pass.

```
  analytics.annotation                0
  analytics.annotator                 0
  analytics.aspect_sentiment       2659
  analytics.feedback_topic         9905
  analytics.nlp_model                 4
  analytics.quality_flag            453
  analytics.text_sentiment         8492
  analytics.topic                    10
  analytics.topic_term              100
  staging.raw_survey               1500
  survey.aspect                       6
  survey.aspect_rating             7150
  survey.crop                       122
  survey.district                     1
  survey.info_source                 12
  survey.interview                 1498
  survey.respondent                1500
  survey.respondent_crop           2125
  survey.scheme                      11
  survey.scheme_alias                91
  survey.scheme_awareness          3276
  survey.scheme_feedback           1231
  survey.village                    140
Loaded 91 scheme spellings, 8492 text scores, 2659 aspect scores, 9905 document-topic weights, 453 quality fla
Verification: 41/41 checks passed -> database_verification.csv
```

Figure 6.1 — Loading the database: row counts and "Verification: 41/41 checks passed"

41 / 41	4 / 4	13.7 s	16 MB
checks passed	bad inserts rejected	full rebuild	database size

Validation inside the database

The human labels are loaded into `analytics.annotation`, and the view `reporting.v_validation_metrics` computes each stored model's accuracy directly in SQL:

```
SELECT * FROM reporting.v_validation_metrics;
          model_name           | n_labelled | n_agree_with_human | accuracy_pct
-------------------------------+------------+--------------------+--------------
 VADER + agri lexicon          |        243 |                 57 |         23.5
 RoBERTa sentiment (zero-shot) |        243 |                123 |         50.6
(2 rows)


SELECT count(*) AS human_labels, count(DISTINCT feedback_id) AS answers FROM analytics.annotation;
 human_labels | answers
--------------+---------
          517 |     346
(1 row)
```

Figure 6.2 — psql: model accuracy against the human labels, computed by a SQL view

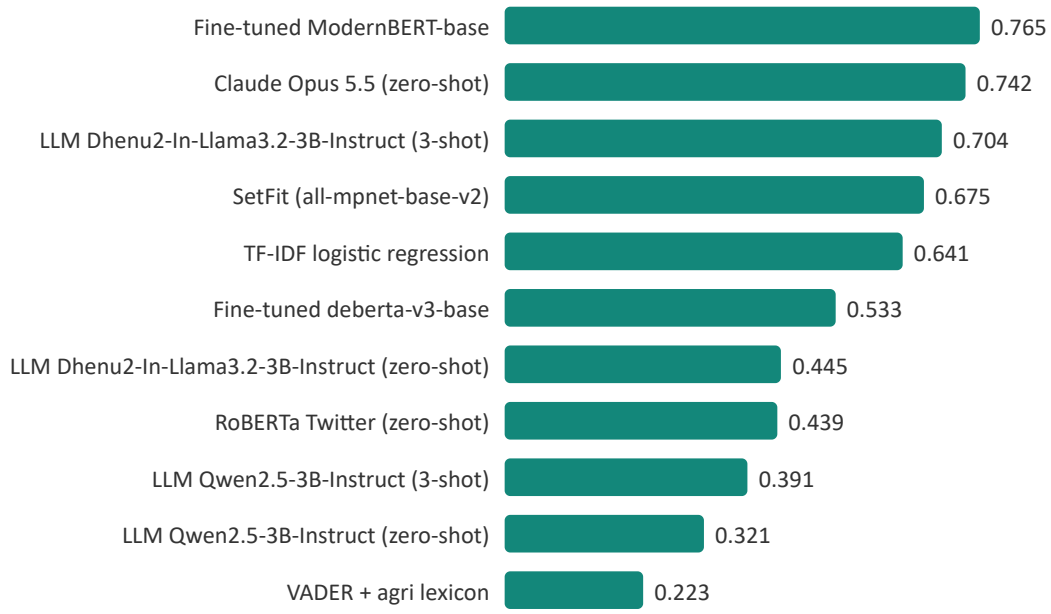
6.2 Model comparison (sentiment)

All models were scored on the same 243 human-labelled test answers. Macro-F1 gives equal weight to positive, neutral and negative answers, so it is fairer than accuracy when classes are unequal.

Model	Type	Accuracy	Macro-F1	Macro-F1 95% CI	Mean ± SD over seeds	Kappa vs labels
Fine-tuned ModernBERT-base	trained (fine-tuned)	0.835	**0.765**	0.687 - 0.832	0.750 ± 0.034	0.683

Model	Type	Accuracy	Macro-F1	Macro-F1 95% CI	Mean ± SD over seeds	Kappa vs labels
Claude Opus 5.5 (zero-shot)	LLM (prompted, external)	0.786	0.742	0.676 - 0.801	single run	0.620
LLM Dhenu2-In-Llama3.2-3B-Instruct (3-shot)	LLM (prompted)	0.749	0.704	0.63 - 0.766	single run	0.566
SetFit (all-mpnet-base-v2)	trained (few-label)	0.840	0.675	0.592 - 0.752	0.685 ± 0.022	0.675
TF-IDF logistic regression	trained baseline	0.745	0.641	0.562 - 0.718	single run	0.491
Fine-tuned deberta-v3-base	trained (fine-tuned)	0.749	0.533	0.476 - 0.595	0.584 ± 0.080	0.481
LLM Dhenu2-In-Llama3.2-3B-Instruct (zero-shot)	LLM (prompted)	0.519	0.445	0.38 - 0.51	single run	0.208
RoBERTa Twitter (zero-shot)	zero-shot baseline	0.506	0.439	0.367 - 0.512	single run	0.103
LLM Qwen2.5-3B-Instruct (3-shot)	LLM (prompted)	0.457	0.391	0.317 - 0.458	single run	0.065
LLM Qwen2.5-3B-Instruct (zero-shot)	LLM (prompted)	0.300	0.321	0.268 - 0.376	single run	0.048
VADER + agri lexicon	zero-shot baseline	0.235	0.223	0.163 - 0.279	single run	-0.144

Figure 6.3 — Macro-F1 of every model



Result: Fine-tuned ModernBERT-base performed best (macro-F1 0.765, accuracy 83.5%). The common VADER lexicon was the weakest, showing that general-purpose sentiment tools do not understand farmers' service feedback, while models that learn from labelled farmer answers — or capable LLMs following the labelling guide — do far better. Its lead over Claude Opus 5.5 (zero-shot) and SetFit (all-mpnet-base-v2) is not statistically significant, so these are best read as tied at the top; it is significantly better than every other model. An Indian-agriculture LLM, Dhenu2, clearly beat a general LLM of the same size (Qwen2.5-3B), showing that domain knowledge helps on farmer feedback.

Is the best model significantly better? (exact McNemar test)

Compared with	Best right, other wrong	Other right, best wrong	p-value	Significant (p < 0.05)
Claude Opus 5.5 (zero-shot)	47	35	0.2242	no
LLM Dhenu2-In-Llama3.2-3B-Instruct (3-shot)	46	25	0.017	yes
SetFit (all-mpnet-base-v2)	13	14	1.0	no
TF-IDF logistic regression	33	11	0.0013	yes
Fine-tuned deberta-v3-base	28	7	0.0005	yes
LLM Dhenu2-In-Llama3.2-3B-Instruct (zero-shot)	99	22	0.0	yes
RoBERTa Twitter (zero-shot)	101	21	0.0	yes
LLM Qwen2.5-3B-Instruct (3-shot)	109	17	0.0	yes
LLM Qwen2.5-3B-Instruct (zero-shot)	156	26	0.0	yes
VADER + agri lexicon	162	16	0.0	yes

Learning curve — how much labelled data is needed

Method	Training answers	Macro-F1	SD
TF-IDF logistic regression	100	0.494	± 0.019
TF-IDF logistic regression	200	0.567	± 0.013
TF-IDF logistic regression	300	0.633	± 0.021
TF-IDF logistic regression	400	0.641	± 0.000

6.3 Aspect-level sentiment

Method	Aspect mentions (human)	Detected	Accuracy	Macro-F1
Keyword ABSA + VADER (current pipeline)	274	201	0.343	0.373
DeBERTa ABSA (deberta-v3-base-absa-v1.1)	274	274	0.522	0.433

The original keyword method detects only some aspect mentions and is often wrong about their polarity, while the purpose-built DeBERTa ABSA model handles every mention. Aspect-level conclusions in this report that rely on the keyword method should therefore be read as indicative.

6.4 Policy findings from the database

Finding 1 — knowing about a scheme is not the same as joining it

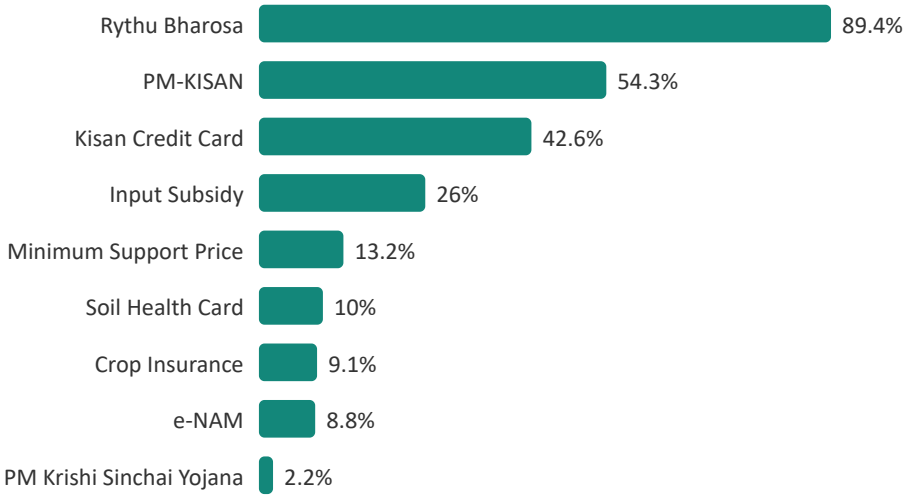


Figure 6.4 — Enrolled, among farmers who know the scheme (reporting.v_awareness_gap)

89.4% of farmers who know Rythu Bharosa have enrolled, but only 9.1% of the 143 farmers who know Crop Insurance have. This comes directly from survey answers, so it does not depend on any NLP model.

Finding 2 — ratings and farmers' own words often disagree

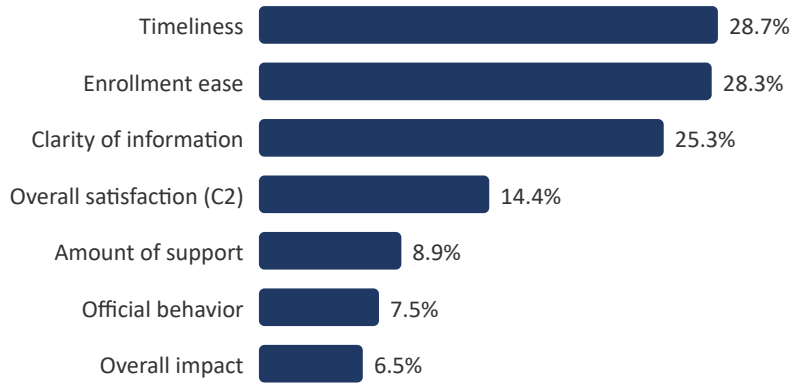


Figure 6.5 — How often a rating contradicts the farmer's own comment, by aspect (reporting.v_aspect_divergence)

Disagreement is highest for timeliness (28.7%) and enrolment (28.3%). The overall rating-versus-text difference is statistically significant (McNemar–Bowker $\chi^2 = 192.73$, $p < 0.001$). Because these aspect labels come from the keyword method, the exact percentages will be recomputed with the best validated model for the research paper.

Finding 3 — enrolment is the weakest aspect of every major scheme

Scheme	Weakest aspect	% negative mentions
Input Subsidy	Enrollment ease	53.8%
Kisan Credit Card	Enrollment ease	50.0%
PM-KISAN	Enrollment ease	47.6%
Rythu Bharosa	Enrollment ease	40.6%

6.5 Performance observations

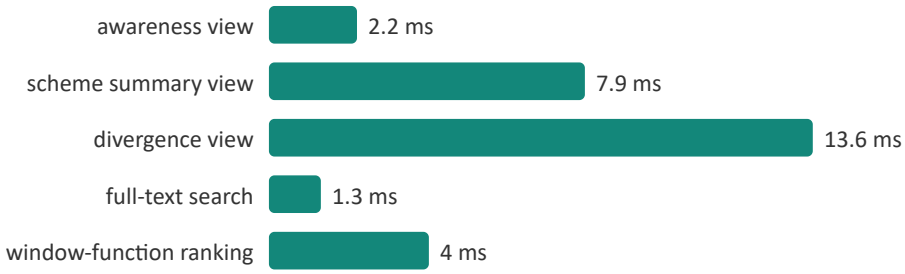


Figure 6.6 — Query time in milliseconds (average of 5 runs)

The whole database rebuilds and verifies in about 14 seconds and every reporting query runs in under 15 ms, so materialised views were not needed at this size. On the model side, the GPU (RTX 2050, 4-bit quantisation) made 3-billion-parameter LLMs practical on a laptop.

7. Challenges and Solutions

Type	Challenge	Solution
Data handling	The same scheme typed 93 different ways	RapidFuzz fuzzy matching plus an alias table recording every match and its score
Data handling	Several answers packed into one cell; one option itself contained a comma	parsed against the questionnaire's option list and stored as rows in link tables
Data handling	355 farmers gave contradictory answers	kept the data and recorded quality flags instead of deleting
Data handling	Answers like "no problems" were scored as negative	treated empty-meaning answers as empty before scoring
Database design	Six rating columns side by side	normalised into one row per feedback and aspect
Database design	Results from many models in one place	nlp_model table; every score carries its model id
Database design	Keeping SQL and Python results consistent	41 automatic checks after every load
Technical	PostgreSQL download blocked (HTTP 403)	downloaded the official installer, verified its signature, installed silently
Technical	LLMs too large for the CPU and laptop memory	CUDA PyTorch on the RTX 2050 with 4-bit quantisation
Technical	Long model runs could be lost on a crash	predictions cached per model so runs resume and combine
Methodology	Interviews in Telugu, notes recorded in English	English models used; noted as a limitation
Methodology	Topic models changed between runs	reported as exploratory only

8. Conclusion and Future Enhancements

8.1 What was achieved

- A normalised PostgreSQL database (4 schemas, 22 tables, 26 foreign keys, 25 CHECK rules, 6 views) holding 40,286 rows, rebuilt and verified automatically.
- An NLP layer whose results are stored with full provenance, and a fair comparison of 11 approaches — from a lexicon to fine-tuned transformers and LLMs including an Indian agriculture model.
- Policy answers in SQL: the awareness–enrolment gap, rating–comment disagreement and the weakest aspect of each scheme.
- Reports, a policy brief, a dashboard dataset and a research paper draft, all generated from the database.

8.2 Learning outcomes

- Turning messy real-world data into a clean relational design (1NF → 3NF).
- Using keys, CHECK constraints, transactions, views, window functions and full-text search in practice.
- Connecting Python and PostgreSQL safely with parameterised queries.
- Evaluating NLP models properly: held-out test sets, multiple seeds, confidence intervals and significance tests.

8.3 Limitations

- One district and one survey round.
- Interviews were in Telugu but recorded in English, so some nuance may be lost.
- Labels were produced jointly as one consensus set, so inter-annotator agreement could not be measured.
- Aspect findings from the keyword method are indicative; topic models are exploratory.

8.4 Future improvements

- Recompute the aspect findings with the best validated model and add independent double-labelling.
- Connect Power BI live to PostgreSQL for an interactive dashboard.
- Extend to more districts and survey rounds; host the database in the cloud.
- Test multilingual models directly on Telugu answers.

8.5 Research publication

The findings — the gap between what farmers rate and what they write, the awareness–enrolment gap, and a validated comparison of NLP models including an Indian agriculture LLM — are being written up as a **research paper targeting a Q1 journal**, under the guidance of Prof. Sangeetha S.

Appendix — Source Code and How to Run

GitHub repository: https://github.com/singhkrish2006/farmer-scheme-policy-intelligence

The repository contains the database schema and queries, all Python modules and scripts, the Colab notebook, this report and the presentation. The raw dataset and the labelling worksheets are not included because they contain farmers' personal details.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python scripts/01_run_pipeline.py          # clean + sentiment
python scripts/02_run_analysis.py          # quality, statistics, topics
python scripts/07_load_database.py         # build PostgreSQL + 41 checks
python scripts/05_evaluate_models.py       # accuracy vs human labels
python scripts/12_compare_models.py        # full model comparison
```

Selected analysis queries (db/queries.sql)

```
-- 1. Awareness-to-enrolment conversion per scheme
SELECT * FROM reporting.v_awareness_gap ORDER BY heard DESC;

-- 4. Most negatively described aspect for each scheme (window function)
WITH aspect_negative AS (
    SELECT s.scheme_name, a.aspect_name,
           round(100.0 * count(*) FILTER (WHERE x.label = 'Negative') / count(*), 1) AS pct_negative,
           count(*) AS mentions
    FROM analytics.aspect_sentiment x
    JOIN survey.scheme_feedback f USING (feedback_id)
    JOIN survey.scheme s USING (scheme_id)
    JOIN survey.aspect a USING (aspect_id)
    GROUP BY s.scheme_name, a.aspect_name
    HAVING count(*) >= 10
)
SELECT scheme_name, aspect_name, pct_negative, mentions
FROM (
    SELECT *, rank() OVER (PARTITION BY scheme_name ORDER BY pct_negative DESC) AS rnk
    FROM aspect_negative
) ranked
WHERE rnk = 1
ORDER BY pct_negative DESC;

-- 6. Full-text search: complaints mentioning delays in documents or verification
SELECT f.feedback_id, s.scheme_name, f.problems_text
FROM survey.scheme_feedback f
JOIN survey.scheme s USING (scheme_id)
WHERE to_tsvector('english', coalesce(f.positive_text, '') || ' ' || coalesce(f.problems_text, '') || '
' || coalesce(f.suggestions_text, ''))
      @@ to_tsquery('english', 'delay & (document | verification)')
LIMIT 20;

-- 8. Scheme-name spellings mapped to each scheme
SELECT s.scheme_name, count(*) AS spellings, string_agg(a.alias_text, ' | ' ORDER BY a.alias_text) AS
examples
FROM survey.scheme_alias a
JOIN survey.scheme s USING (scheme_id)
GROUP BY s.scheme_name
ORDER BY spellings DESC;

-- 10. Model accuracy against human labels (after annotation is loaded)
```

```
SELECT * FROM reporting.v_validation_metrics;

-- 11. Trace any respondent back to the raw survey row
SELECT r.respondent_id, r.source_excel_row, r.name, raw.c1_scheme, raw.c4_positive, raw.c5_problems
FROM survey.respondent r
JOIN staging.raw_survey raw USING (source_excel_row)
WHERE r.respondent_id = 42;
```